

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 4 Data transfer instructions - REAL numbers

The FPU instructions covered in this chapter perform no mathematical operation on numerical data. Their main purpose is simply to transfer floating point data between the FPU's 80-bit data registers, or between those registers and memory, or to access the 7 constants hard coded into the FPU. (*Other instructions must be used to transfer [integer](#) and [BCD](#) data*).

The data transfer instructions are (in alphabetical order):

FCMOVcc	Conditional MOVe based on CPU flags
FLD	LoaD real number
FST	STore real number
FSTP	STore real number and Pop the top data register
FXCH	eXCHange the top data register with another data register

The instructions to transfer the 7 hard-coded constants into the top data register are (in alphabetical order):

FLD1	LoaD the value of 1
FLDL2E	LoaD the Log base 2 of e (Napierian constant)
FLDL2T	LoaD the Log base 2 of Ten
FLDLG2	LoaD the Log base 10 of 2 (common log of 2)
FLDLN2	LoaD the Log base e of 2 (natural log of 2)
FLDPI	LoaD the value of PI
FLDZ	LoaD the value of Zero

FLD (Load real number)

Syntax: **fld Src**

Exception flags: Stack Fault, Invalid operation, Denormalized value

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the floating point value from the specified source (*Src*) in the new TOP data register. The source can be one of the FPU's data registers or the memory address of a REAL4, REAL8, or REAL10 value (see Chap.2 for [addressing modes of real numbers](#)).

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid operation** exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

If the source is a [denormalized](#) number, a **Denormal** exception will be detected, setting the related flag in the Status Word. The value will still be loaded and normalized if possible.

If the source is one of the FPU's data registers, its value is retrieved before the TOP register pointer is decremented. If that source register is empty, both a **Stack Fault** and an **Invalid operation** exceptions are

detected, setting both flags in the Status Word. The TOP register pointer will still be decremented and the new value in ST(0) will be the INDEFINITE NAN.

The content of the [Tag Word](#) will be modified according to the result of the operation.

Examples of using this instruction:

```
fld st(3) ;the value in the current ST(3) is copied into the new ST(0)
           ;the same value will now be in ST(0) and ST(4)
```

```
fld st ;duplicates the TOP value which will now be in both ST(0) and ST(1)
```

Note that **fld st(7)** will always be an Invalid operation and the outcome will always be the INDEFINITE value in the new ST(0). *The ST(7) register must be empty for a valid FLD instruction but would also be an invalid source if it is empty!*

```
fld real4_var ;the value of the real4_var variable
                ;is converted to the REAL10 format and copied into ST(0)
```

```
fld qword ptr[ebx] ;ST(0) takes the value of the QWORD located at the address
                    ;indicated by EBX which the FPU interprets as a REAL8 value
```

```
fld tbyte ptr[edi+50] ;the FPU interprets the value at the address EDI+50
                       ;as a REAL10 and copies it to ST(0).
```

The content of memory is simply a series of bits. Floating point data cannot be differentiated from any other type of data. The FPU will thus interpret a series of bits as instructed. (See [Rule #3](#)).

If the source is a REAL10 value in memory, it has the same format as the FPU's data registers and is simply copied as is.

However, if the source is a [REAL4](#) or a [REAL8](#) value, it is converted to the 80-bit [REAL10](#) format before loading it to ST(0). When such an operation is observed with a debugger, the displayed value in the FPU data register may often seem slightly different from the intended one. That would be due to the rounding which occurred when the data had to be shortened to fit into the REAL4 or REAL8 format, and some precision was lost.

Just as 3.33333... is never the exact value of $10\frac{1}{3}$ regardless of the length of the fractional part, binary fractions behave in a similar manner. For example, a decimal value of 1.2 in binary would be:

```
1.00110011001100110011001100110011....b    0011 repeating indefinitely.
```

If a variable is declared as a REAL4 and initialized with that value, the assembler would most probably have used the FPU to convert it to binary with 64 bits of precision using the "0011" sequence as often as possible. To fit that value into the REAL4 format, it could only keep the first 23 fraction bits. The 24th bit being a "1", the number was rounded UP to:

```
1.0011 0011 0011 0011 0011 010b
```

Because of this rounding UP, this value is now slightly larger than the original decimal value and a debugger would show it as follows if loaded into an FPU data register:

```
1.2000000476837158
```

Similarly, some binary fractions would be rounded DOWN and the resulting value would be slightly smaller than the original one. For example, a REAL4 variable initialized with a decimal value of 1.3 would be loaded to the FPU's data register as:

```
1.0100 1100 1100 1100 1100 110b
```

which would be shown in decimal format by a debugger as:

```
1.2999999523162842
```

This rounding can also affect REAL4 variables initialized with large numbers having more than 7 integer digits. For example, if a REAL4 variable is initialized with 987654321.0 and loaded to the FPU, a debugger would show it as 987654336.0!

The FPU and floating point real numbers do not provide unlimited precision. The REAL4 format's relative precision is only equivalent to approx. 7 significant decimal digits, the REAL8 format being equivalent to approx. 15 significant decimal digits, and the REAL10 to 19 significant decimal digits.

FST (Store real number)

Syntax: **fst Dest**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Precision

This instruction stores the value currently in the TOP data register ST(0) to the specified destination (*Dest*). The destination can be one of the FPU's data registers or the memory address of a REAL4 or REAL8 value (see Chap.2 for [addressing modes of real numbers](#)).

This instruction cannot store the value of ST(0) as a REAL10 in memory; see the [FSTP](#) instruction for such an operation.

If the ST(0) data register is empty, both a Stack Fault and an Invalid operation exceptions are detected, setting both flags in the [Status Word](#), and the value of [INDEFINITE](#) would be stored at the specified destination. If the ST(0) contains a [denormalized](#) number, a Denormal exception will be detected, setting the related flag in the Status Word.

When the destination is one of the FPU's data registers, the content of ST(0) will overwrite the content of the destination register, whether that destination register is empty or not, and adjust the [Tag Word](#) to reflect any change in the status of that register.

When the destination is a [REAL4](#) or [REAL8](#) memory address, the [REAL10](#) value in ST(0) is first converted to the appropriate format and rounded according to the bits in the **RC** field of the [Control Word](#). A Precision exception is detected if some of the least significant bits would be lost, setting the related flag in the Status Word. A Denormal exception may also be detected, setting the related flag in the Status Word, if the value of ST(0) must be denormalized to fit the destination's format.

Examples of using this instruction:

```
fst st(3)    ;the value of the current ST(0) is copied to the current ST(3)
               ;the same value will now be in ST(0) and ST(3)
```

```
fst st       ;although this instruction is allowed, no data register is changed
               ;an exception may still be detected due to the value in ST(0)
```

*Note that **fst st(7)** is valid but is not recommended because it would prevent loading other values to the FPU. If for some reason it should become necessary to use it, ST(7) should be emptied as soon as possible (such as with the [FFREE](#) instruction).*

```
fst dword ptr [eax] ;stores the value of ST(0) in the REAL4 format
                     ;at the memory address indicated by EAX
```

```
fst real_var      ;stores the value of ST(0) in the appropriate format
                     ;at the address of the real_var memory variable
```

Although the FPU performs computations with 64 bits of precision, 40 of those bits of precision will be lost when a computed value is stored as a REAL4. Re-using such a stored value in subsequent computations will automatically limit the precision to 24 bits even though 64 bits would be used.

Similarly, 11 bits of precision will be lost when a computed value is stored as a REAL8. Re-using such a stored value in subsequent computations will automatically limit the precision to 53 bits even though 64 bits would be used.

FSTP (Store real number and Pop the top data register)

Syntax: **fstp Dest**

Exception flags: Stack Fault, Invalid operation, Denormalized value,
Precision

This instruction is the same as the [FST](#) instruction except for the following two additions:

- it allows storing the ST(0) value in memory as a [REAL10](#) value, and
- the ST(0) register is [POPped](#) after the transfer is completed, modifying the [Tag Word](#) and incrementing the TOP register pointer of the [Status Word](#).

This instruction would be used, for example,

- when the final result of a computation must be stored in memory, liberating the FPU's TOP data register for future use,

fstp qword ptr [edx] ;stored as a REAL8 at the address specified by EDX

- when an intermediate result of a computation in ST(0) must be preserved without any loss of precision,

fstp temp_var ;temp_var having been declared as a TBYTE

- when the reuse of a data register could be beneficial while liberating the TOP register,

fstp st(4) ;after the instruction is completed, the former value of ST(4)
;would be overwritten by the former value of ST(0), and
;would become ST(3) after popping ST(0)

- when the value in ST(0) has become useless and the register needs to be liberated.

fstp st ;copied upon itself and popped

FXCH (Exchange the top data register with another data register)

Syntax: **fxch Dest**
fxch (no operand, ST(1) being implied)

Exception flags: Stack Fault, Invalid operation

This instruction exchanges the content of the TOP data register ST(0) with the content of one of the other data registers (*Dest*). (*Memory operands are not allowed with this instruction.*)

If either ST(0) or the destination register is empty, both a **Stack Fault** and an **Invalid operation** exceptions are detected, setting both flags in the [Status Word](#). The value of [INDEFINITE](#) will have been assumed in the empty register and then exchanged with the value of the other register. The [Tag Word](#) will be adjusted to reflect any change in the status of the two registers involved.

With many FPU instructions, the data in ST(0) must be one of the operands or even the only operand. FXCH is useful when some operation on data other than in the ST(0) register may be required during a computation. For example, if the square root of the value in ST(3) should be needed before proceeding with a computation, the following code could be used:

```
fxch st(3) ;get the value of ST(3) into the TOP register
            ;the value of ST(0) being kept in ST(3)
fsqrt      ;extract its square root
```

fxch st(3) ;return it to its relative position and
;bring back the former value of ST(0) to its original position

FCMOVcc (Conditional move based on CPU flags)

Syntax: **fcmovcc st,st(i)**

Exception flags: Stack Fault, Invalid operation

Note: This instruction is valid only for the Pentium Pro and subsequent processors. It may not be supported by some assemblers (for MASM, the .686 directive must be used). The encodings are provided to facilitate hard-coding of this instruction if it is not supported by the assembler.

This instruction overwrites the content of the TOP data register ST(0) with the content of the specified ST(i) data register if the specified condition is true. Following is the list of the conditions supported with the instruction, with the encodings and descriptions.

Encoding	Instruction	Description
DA C0+i	FCMOVB ST,ST(i)	Move if below (CF=1)
DA C8+i	FCMOVE ST,ST(i)	Move if equal (ZF=1)
DA D0+i	FCMOVBE ST,ST(i)	Move if below or equal (CF=1 or ZF=1)
DA D8+i	FCMOVU ST,ST(i)	Move if unordered (PF=1)
DB C0+i	FCMOVNB ST,ST(i)	Move if not below (CF=0)
DB C8+i	FCMOVNE ST,ST(i)	Move if not equal (ZF=0)
DB D0+i	FCMOVNBE ST,ST(i)	Move if not below or equal (CF=0 and ZF=0)
DB D8+i	FCMOVNU ST,ST(i)	Move if not unordered (PF=0)

If the ST(i) data register is empty, both a **Stack Fault** and an **Invalid operation** exceptions are detected, setting both flags in the [Status Word](#). The value of [INDEFINITE](#) will have been assumed in the empty register and overwrite the content of the TOP data register ST(0).

Extreme caution must be used with this instruction depending on how the CPU flags have been modified. For example, when PF=1 due to an FPU instruction generating an invalid operation exception, the CF and ZF flags are also usually set to 1. If the instruction is used after the CPU flags are modified by a regular CPU instruction, the PF flag may not have any significant meaning.

FLDZ (Load the value of Zero)

Syntax: **fldz** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of +0.0 into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid operation** exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

A typical use of this instruction would be to "initialize" a data register intended to be used as an accumulator. Even though a value of zero could also be easily loaded from memory, this instruction is faster and does not need the use of memory.

FLD1 (Load the value of 1)

Syntax: **fld1** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of +1.0 in [REAL10](#) format into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

A value of 1 may be required in some computations. One example would be when the reciprocal of a number is needed. Even though a value of 1 could also be easily loaded from memory, this instruction is faster and does not need the use of memory.

FLDPI (Load the value of PI)

Syntax: **fldpi** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of π (3.14159...) in [REAL10](#) format with a precision equivalent to approximately 19 decimal digits into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

The value of π is often required for computations related to the circle, sphere, trigonometry, converting angles to/from degrees/radians, and numerous other applications.

FLDL2E (Load the Log base 2 of the Napierian constant e)

Syntax: **fldl2e** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of $\log_2(e)$ in [REAL10](#) format with a precision equivalent to approximately 19 decimal digits into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

This constant is most useful when an exponential of e must be computed, such as for the hyperbolic functions. From logarithmic relations:

$$\log_2(e^y) = y \log_2(e)$$

The antilog base 2 of the $y \log_2(e)$ result can then be computed using other FPU instructions (including [F2XM1](#)) to arrive at the e^y value.

FLDL2T (Load the Log base 2 of 10)

Syntax: **fldl2t** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of $\log_2(10)$ in [REAL10](#) format with a precision equivalent to approximately 19 decimal digits into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

This constant is most useful when an exponential of 10 (such as the common antilog) must be computed. From logarithmic relations:

$$\log_2(10^y) = y \log_2(10)$$

The antilog base 2 of the $y \log_2(10)$ result can then be computed using other FPU instructions (including [F2XM1](#)) to arrive at the 10^y value.

FLDLG2 (Load the log base 10 of 2)

Syntax: **fldlg2** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of $\log_{10}(2)$ in [REAL10](#) format with a precision equivalent to approximately 19 decimal digits into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

Although this constant is the reciprocal of the $\log_2(10)$ constant obtained with the [FLDL2T](#) instruction, it is preferred for computing the common log (base 10) of numbers because it would involve a multiplication instead of a division according to the following logarithmic relations:

$$\log_{10}(x) = \log_{10}(2) \cdot \log_2(x)$$

A $\log_2(x)$ value can be obtained with the [FYL2X](#) or [FYL2XP1](#) instructions.

Computing the common log of a number may have many uses. One of them is to get information on the relative size of a real number before proceeding to convert it and display it in decimal format, whether it be in regular or scientific notation.

FLDLN2 (Load the log base *e* of 2)

Syntax: **fldln2** (no operand)

Exception flags: Stack Fault, Invalid operation

This instruction decrements the **TOP** register pointer in the [Status Word](#) and loads the value of $\ln(2)$ in [REAL10](#) format with a precision equivalent to approximately 19 decimal digits into the new TOP data register.

If the ST(7) data register which would become the new ST(0) is not empty, both a **Stack Fault** and an **Invalid** operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the [INDEFINITE](#) NAN.

Although this constant is the reciprocal of the $\log_2(e)$ constant obtained with the [FLDL2E](#) instruction, it is preferred for computing the natural log (base e) of numbers because it would involve a multiplication instead of a division according to the following logarithmic relations:

$$\ln(x) = \ln(2) \cdot \log_2(x)$$

A $\log_2(x)$ value can be obtained with the [FYL2X](#) or [FYL2XP1](#) instructions.

The natural log of a number is often used in some scientific computations. For example, it is necessary for computing the hyperbolic arc functions.

[RETURN TO
SIMPLY FPU
CONTENTS](#)